

HOMEWORK 6 · FINAL CAPSTONE

AI-Powered Transaction Processing Pipeline

A file-based pipeline with fraud detection, a React dashboard, a custom MCP server, and a coverage-gated test suite.

Created by Mykola Bernadskyi

What it does

- ▶ Ingests raw bank transactions and processes each through **3 sequential stages**
- ▶ **Validation** → **Fraud Detection** → **Settlement**, with a full audit trail
- ▶ Stages hand off data as **JSON files** through shared directories — no direct calls
- ▶ Money is **decimal.Decimal** everywhere; account numbers treated as PII, never logged
- ▶ Observed via a **React dashboard**; queryable via a custom **MCP server**

ARCHITECTURE

File-based pipeline + REST + MCP

```
React (Vite) —HTTP→ FastAPI → orchestrator.py
                        |
sample-transactions.json —┐
                        |
input → Validator → processing → Fraud → output → Settlement → results
                        |
                        FastMCP server reads ←┐
                        (get_transaction_status, pipeline://summary)
```

Every hop writes a valid JSON envelope · shared/ is the hand-off medium

Three stages, one contract

- ▶ **Validator** — required fields, Decimal amount (negative only for refunds), ISO 4217 currency; rejects with a reason
- ▶ **Fraud Detector** — additive risk score 0.0–1.0: high-value +0.5, off-hours +0.3, cross-border +0.2, wire +0.1; flags ≥ 0.5
- ▶ **Settlement** — 0.5% fee + net with ROUND_HALF_UP; preserves the fraud flag; rejected records never settle

Each stage is a pure function: `process_transaction(record)` → `record`

Outcomes

8

Total processed

4

Settled

3

Flagged

1

Rejected

- ▶ **TXN002 / TXN005** — \$25k / \$75k wires → flagged (high-value + wire)
- ▶ **TXN004** — 02:47 UTC, Germany → flagged (off-hours + cross-border)
- ▶ **TXN006** — currency "XYZ" → rejected (not ISO 4217)
- ▶ **TXN007** — −\$100 refund → allowed (negative permitted for refunds)

Stack & quality gate

Pipeline	Python 3.12 · Decimal
API	FastAPI + uvicorn
Frontend	React + TS + Vite
MCP server	FastMCP 3
Tests	pytest + pytest-cov

98%

Test coverage (56 tests) · gate floor 80%

- ▶ Coverage gate **blocks push** below 80%
- ▶ git pre-push + Claude Code hook

Lessons learned

- ▶ **Spec first.** Writing specification.md before code made every stage boundary obvious
- ▶ **Files are the persistence.** agents.md + shared/ let any session (or stage) resume cleanly
- ▶ **context7 beat memory.** Pulling live docs for Decimal & CORS avoided stale-API guesswork
- ▶ **The hook is the point.** A machine-enforced coverage gate is worth more than good intentions
- ▶ **Determinism matters** in finance — the same transaction must never be judged twice differently

Thank you — Mykola Bernadskyi